



CHƯƠNG 5

QUẢN LÝ GIAO TÁC

NỘI DUNG



GIAO TÁC

CÁC VẤN ĐỀ CỦA TRUY XUẤT ĐỒNG

KỸ THUẬT KHÓA



GIAO TÁC LÀ GÌ?

KHÁI NIỆM GIAO TÁC



- ❑ Giao tác (transaction) là 1 đơn vị xử lý nguyên tố gồm 1 chuỗi các hành động tương tác lên cơ sở dữ liệu.
- ❑ Khi thực hiện một giao tác thì phải thực hiện tất cả các hành động của nó hoặc không thực hiện bất kỳ hành động nào.

MỘT SỐ THUẬT NGỮ



- ❑ Begin tran[saction]: bắt đầu một giao tác.
- ❑ Commit tran[saction]: hoàn tất một giao tác.
- ❑ Rollback tran[saction]: quay lui, hủy bỏ toàn bộ phần giao tác đã thực hiện trước đó.

TÍNH CHẤT **ACID** CỦA GIAO TÁC



1. Tính nguyên tố (**A**tomicity)
2. Tính nhất quán (**C**onsistancy)
3. Tính độc lập (**I**solation)
4. Tính bền vững (**D**urability)

TÍNH NGUYÊN TỔ (ATOMICITY)



- ❑ Một giao tác có nhiều hành động thì hoặc tất cả các hành động hoặc không có bất kỳ hành động nào được hoàn thành.
- ❑ Ví dụ: Việc chuyển tiền gồm 2 hành động: trừ tiền của tài khoản A và cộng tiền vào tài khoản B. Trong quá trình thực hiện nếu tài khoản A chưa trừ tiền thì tài khoản B cũng chưa được cộng số tiền tương ứng.

TÍNH NHẤT QUẢN (CONSISTANCY)



- ❑ Một giao tác hoặc là sẽ tạo ra một trạng thái mới và hợp lệ cho dữ liệu, hoặc trong trường hợp có lỗi sẽ chuyển toàn bộ dữ liệu về trạng thái trước khi thực thi giao tác.
- ❑ Ví dụ: Việc chuyển tiền gồm 2 hành động: trừ tiền của tài khoản A và cộng tiền vào tài khoản B. Giả sử hành động trừ tiền tài khoản A đã được thực hiện, hành động cộng tiền vào tài khoản B gặp lỗi. Khi đó cả 2 hành động đều bị hủy bỏ và quay lui (rollback) về trạng thái ban đầu.

TÍNH ĐỘC LẬP (ISOLATION)



- ❑ Một giao tác không ảnh hưởng hoặc không chịu ảnh hưởng bởi các giao tác khác.

TÍNH BỀN VỮNG (DURABILITY)



- ❑ Sau khi giao tác được hoàn thành, tất cả những thay đổi trên CSDL do giao tác này thực hiện phải được ghi nhận chắc chắn (vào ổ cứng).
- ❑ Các hệ quản trị CSDL luôn phải có cơ chế phục hồi dữ liệu để đảm bảo điều này, thường dùng cơ chế ghi nhận bằng transaction log

VÍ DỤ VỀ GIAO TÁC



LOP (MALOP, TENLOP, SISO)

SINHVIEN(MASV, HOTEN, TUOI, MALOP)

YC: Khi thêm một sinh viên mới trên SINHVIEN phải cập nhật lại SISO trên LOP.

```
BEGIN TRAN
  Insert into SINHVIEN
  Values ('sv003', 'Mai Lan', '18', '09DHTH3')

  Update LOP
  Set SISO=SISO + 1
  Where MALOP='09DHTH1'
COMMIT TRAN
```

CÁC VẤN ĐỀ CỦA TRUY XUẤT ĐỒNG THỜI



- ❖ Mất dữ liệu đã cập nhật (updated lost)
- ❖ Đọc phải dữ liệu rác (dirty read)
- ❖ Không thể đọc lại (unrepeated read)
- ❖ Bóng ma dữ liệu (phantom problem)

MẤT DỮ LIỆU ĐÃ CẬP NHẬT



- ❖ Là tình trạng mà hành động cập nhật dữ liệu của giao tác P_k bị hành động cập nhật của giao tác P_h ghi đè dẫn đến dữ liệu cập nhật của T_k bị mất.
- ❖ Ví dụ: Xét 2 giao tác P_1 và P_2 xử lý đồng thời, tại thời điểm ban đầu $A=50$:

Thời điểm	Giao tác P_1	Giao tác P_2
t_1	Read(A) (50)	
t_2		Read(A) (50)
t_3	$A=A-10$ (40)	
t_4		$A=A+30$ (80)
t_5	Write(A) (40)	
t_6		Write(A) (80)
t_7	Read(A) (80)	

ĐỌC PHẢI DỮ LIỆU RÁC



- ❖ Là tình trạng một giao tác P_k đọc vùng giá trị A được cập nhật bởi giao tác P_h . Tuy nhiên vì lý do nào đó, giao tác P_h bị hủy dẫn đến vùng dữ liệu A quay về giá trị ban đầu. Khi đó giá trị A mà giao tác P_k đã đọc trước đó trở thành giá trị rác.
- ❖ Ví dụ: Xét 2 giao tác P_1 và P_2 xử lý đồng thời, tại thời điểm ban đầu, $A=50$

Thời điểm	Giao tác P_1	Giao tác P_2
t_1		Read(A) (50)
t_2		$A=A+10$ (60)
t_3		Write(A) (60)
t_4	Read(A) (60)	
t_5	Print(A) (60)	
t_6		Rollback

KHÔNG THỂ ĐỌC LẠI



- ❖ Xét 2 giao tác P_1 và P_2 xử lý đồng thời, giá trị A ban đầu là 50. P_1 Read(A) lần 1 có giá trị $A=50$, lần 2 Read(A) lại thấy giá trị $A=60$. Tình trạng này cho thấy giao tác P_1 không thể đọc lại giá trị A như ban đầu.

Thời điểm	Giao tác P_1	Giao tác P_2
t_1	Read(A) (50)	
t_2	Print(A) (50)	
t_3		Read(A) (50)
t_4		$A=A+10$ (60)
t_5		Write(A) (60)
t_6	Read(A) (60)	

BÓNG MA DỮ LIỆU



- ❖ Xét 2 giao tác P_1 và P_2 xử lý đồng thời, ban đầu $A=50$, $B=30$. P_1 Read(>40) lần 1 có giá trị $A=50$, lần 2 Read(>40) lại thấy 2 giá trị $A=50$ và $C=70$.

Thời điểm	Giao tác P_1	Giao tác P_2
t_1	Read(>40) ($A=50$)	
t_2		$C=70$
t_3		Write(C) ($C=70$)
t_4	Read(>40) ($A=50, C=70$)	

Bài tập



1/ Sử dụng transaction để viết lệnh kiểm tra khi người dùng đăng nhập vào hệ thống.

2/ Chọn một CSDL quen thuộc. Trình bày 4 vấn đề truy xuất đồng thời khi tương tác với CSDL trên, minh họa cụ thể.

KỸ THUẬT KHÓA



- ❖ Khái niệm
- ❖ Các mức độ cô lập
- ❖ Khóa trực tiếp trong câu lệnh
- ❖ Deadlock

KỸ THUẬT KHÓA



❖ KHÁI NIỆM:

- Kỹ thuật khóa là một trong những kỹ thuật dùng để giải quyết các vấn đề của truy xuất đồng thời.
- Một giao tác P trước khi muốn thao tác (read/write) lên một đơn vị dữ liệu A phải phát ra một yêu cầu xin khóa A: **lock(A)**
- Nếu yêu cầu được chấp thuận thì giao tác P mới được phép thao tác lên đơn vị dữ liệu A
- Sau khi thao tác xong, giao tác P phải phát ra lệnh giải phóng A: **unlock(A)**

VÍ DỤ KỸ THUẬT KHÓA



Thời điểm	Giao tác P_1	Giao tác P_2
t_1	Lock(A) Read(A)	
t_2	$A=A+10$	
t_3	Write(A)	
t_4	Unlock(A)	Lock(A) Read(A)
t_5		$A=A-20$
t_6		Write(A)
t_7		Unlock(A)

CÁC LOẠI KHÓA



❖ Khóa đọc

- Khóa đọc (Read lock), trong SQL Server gọi là Share lock (Slock).
- Giao tác giữ khóa Slock thì được phép đọc dữ liệu, nhưng không được phép ghi.
- Nhiều giao tác có thể đồng thời giữ Slock trên cùng 1 đơn vị dữ liệu.

CÁC LOẠI KHÓA



❖ Khóa ghi

- Khóa ghi (Write lock), trong SQL Server gọi là Exclusive lock (Xlock).
- Giao tác giữ Xlock được phép ghi và đọc dữ liệu.
- Tại 1 thời điểm chỉ có tối đa 1 giao tác được quyền giữ Xlock trên 1 đơn vị dữ liệu.
- Không thể thiết lập Slock trên đơn vị dữ liệu đang có ở trạng thái Xlock.

MỨC ĐỘ CÔ LẬP CỦA GIAO TÁC



- ❖ Mức độ cô lập được sử dụng để tự động thiết lập khóa cho các thao tác đọc trong kết nối dữ liệu hiện hành.
- ❖ **Các mức độ cô lập:**
 - Read Uncommitted
 - Read Committed
 - Repeatable Read
 - Serializable

Read Uncommitted



□ Đặc điểm:

- **Đọc dữ liệu:** không cần phải thiết lập Slock, có thể đọc dữ liệu đang được cập nhật bởi giao tác khác ngay cả khi dữ liệu chưa commit (uncommitted).
- **Ghi dữ liệu:** SQL Server tự động thiết lập XLock trên đơn vị dữ liệu được ghi, *XLock được giữ cho đến hết giao tác.*

Ví dụ về Read Uncommitted

- Ví dụ 1: Xét trường hợp thông thường khi không dùng Read Uncommitted => xảy ra Updated lost

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM

P1	P2
<pre>UPDATE KHACHHANG SET DIACHI='VUNG TAU' WHERE MAKH='KH001' SELECT DIACHI FROM KHACHHANG WHERE MAKH='KH001'</pre> DA NANG	<pre>UPDATE KHACHHANG SET DIACHI='DA NANG' WHERE MAKH='KH001'</pre>

Ví dụ về Read Uncommitted(tt)

- Ví dụ 2: Sử dụng Read Uncommitted=> Giải quyết Updated lost

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM

P2 không xin được Xlock nên phải chờ đến khi P1 kết thúc giao tác và unlock

P1	P2
BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED</i> (1) Xlock UPDATE KHACHHANG SET DIACHI='VUNG TAU' WHERE MAKH='KH001' WAITFOR DELAY '00:00:10' SELECT DIACHI FROM KHACHHANG WHERE MAKH='KH001' COMMIT TRAN VUNG TAU (2) Xlock	BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED</i> UPDATE KHACHHANG SET DIACHI='DA NANG' WHERE MAKH='KH001' COMMIT TRAN (3) Xlock (4) Xlock

Ví dụ về Read Uncommitted(tt)

- Ví dụ 3: Read Uncommitted=> không giải quyết được Dirty read

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM

P1	P2
BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED</i> (1) Xlock UPDATE KHACHHANG SET DIACHI='VUNG TAU' WHERE MAKH='KH001' WAITFOR DELAY '00:00:10' (3) Xlock ROLLBACK TRAN TPHCM	BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ UNCOMMITTED</i> (2) SELECT DIACHI FROM KHACHHANG WHERE MAKH='KH001' VUNG TAU COMMIT TRAN

Nhận xét Read Uncommitted



❖ Ưu điểm:

- Giải quyết vấn đề Lost Updated
- Không cần thiết lập Slock khi đọc=> không cản trở giao tác khác giữ khóa Xlock.

❖ Hạn chế:

- Có khả năng xảy ra 3 vấn đề của truy xuất đồng thời: Dirty Read, Unrepeatable Read, Phantom

Read Committed



❖ Đặc điểm:

- **Đọc dữ liệu:** SQL server tự động thiết lập SLock trên đơn vị dữ liệu được đọc, SLock được giải phóng ngay sau khi đọc xong.
- **Ghi dữ liệu:** SQL server tự động thiết lập XLock trên đơn vị dữ liệu được ghi, XLock được giữ cho đến hết giao tác.

Ví dụ về Read Committed

- Ví dụ: Read Committed=> Giải quyết được Dirty read

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM

P2 phải chờ đến khi P1 kết thúc giao mới xin được Slock

P1	P2
BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ COMMITTED</i> UPDATE KHACHHANG SET DIACHI='VUNG TAU' WHERE MAKH='KH001' WAITFOR DELAY '00:00:10' ROLLBACK TRAN TPHCM	BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ COMMITTED</i> SELECT DIACHI FROM KHACHHANG WHERE MAKH='KH001' COMMIT TRAN TPHCM

(1) Xlock

(3) Slock

(4) Slock

(2) Xlock

Ví dụ về Read Committed(tt)

- Read Committed=> Không giải quyết được Unrepeatable read

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM

P1	P2
BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ COMMITTED</i> (1) Slock SELECT DIACHI FROM KHACHHANG TPHCM (2) Slock WHERE MAKH='KH001' WAITFOR DELAY '00:00:10' (5) Slock SELECT DIACHI FROM KHACHHANG VUNG TAU (6) Slock WHERE MAKH='KH001' COMMIT TRAN	BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL READ COMMITTED</i> UPDATE KHACHHANG (3) Xlock SET DIACHI='VUNG TAU' WHERE MAKH='KH001' COMMIT TRAN (4) Xlock

Nhận xét Read committed



❖ Ưu điểm:

- Giải quyết vấn đề Dirty Read, Lost Updated
- SLock được giải phóng ngay => không cản trở nhiều đến thao tác ghi dữ liệu của các giao tác khác.

❖ Hạn chế:

- Chưa giải quyết được vấn đề Unrepeatable Read, Phantom

Repeatable read



❖ Đặc điểm:

- **Đọc dữ liệu:** SQL server tự động thiết lập Slock trên đơn vị dữ liệu được đọc và giữ Slock đến hết giao tác.
- **Ghi dữ liệu:** SQL server tự động thiết lập XLock trên đơn vị dữ liệu được ghi, XLock được giữ cho đến hết giao tác.

Ví dụ về Repeatable read

- Repeatable read => Giải quyết được Unrepeatable read

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM

P2 phải chờ đến khi P1 giải phóng Slock mới xin được Xlock

P1	P2
BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL REPEATABLE READ</i> (1) Slock SELECT DIACHI FROM KHACHHANG WHERE MAKH='KH001' WAITFOR DELAY '00:00:10' TPHCM SELECT DIACHI FROM KHACHHANG WHERE MAKH='KH001' TPHCM (2) Slock COMMIT TRAN	BEGIN TRAN <i>SET TRANSACTION ISOLATION LEVEL REPEATABLE READ</i> UPDATE KHACHHANG SET DIACHI='VUNG TAU' WHERE MAKH='KH001' (3) Xlock COMMIT TRAN (4) Xlock

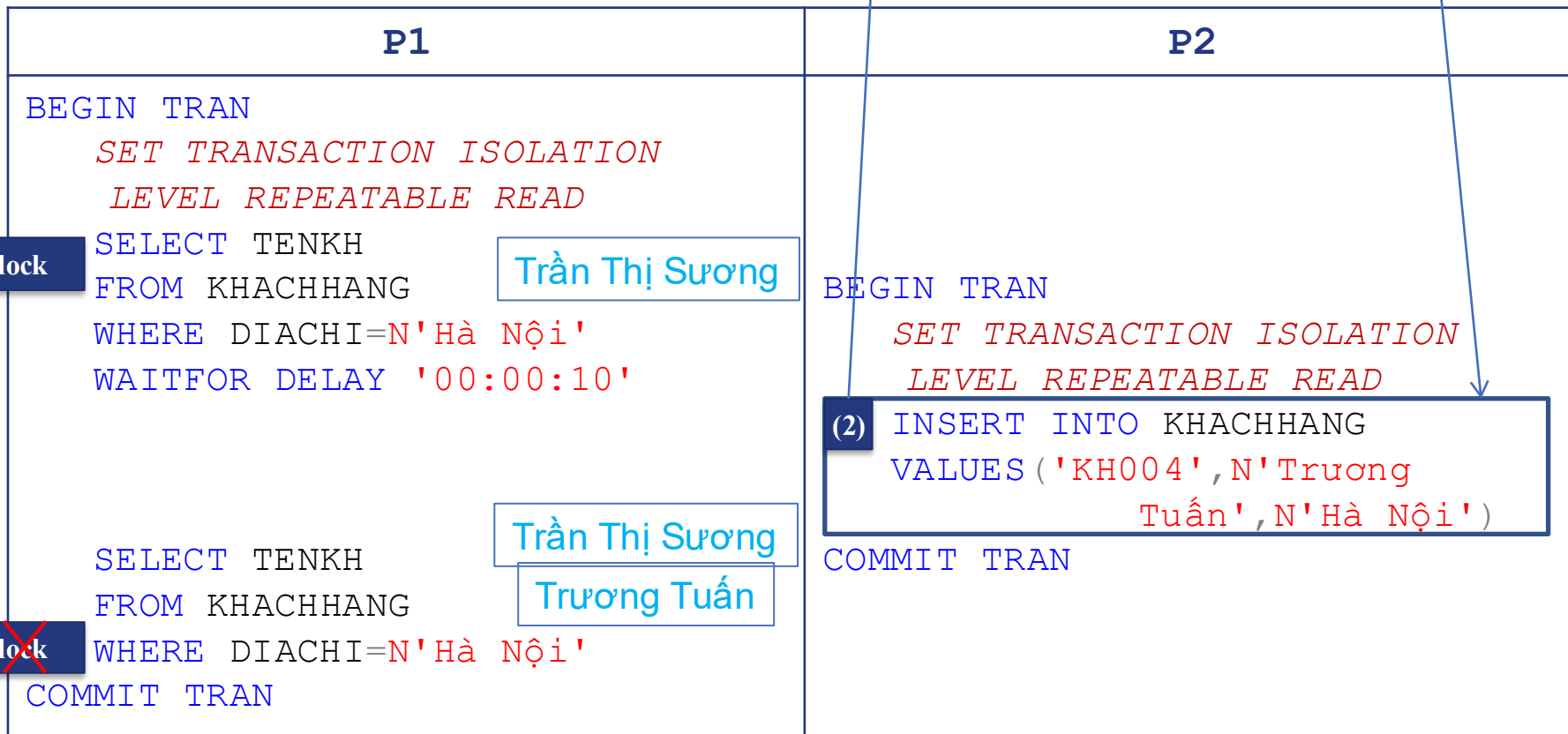
Ví dụ về Repeatable read

- Repeatable read => Không giải quyết được Phantom

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	HA NOI
KH003	Bùi Văn Toàn	TPHCM

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM
KH004	Trương Tuấn	Hà Nội

Repeatable Read chỉ ngăn được lệnh UPDATE nhưng không ngăn được lệnh INSERT hoặc DELETE



Nhận xét Repeatable read

❖ Ưu điểm:

- Giải quyết được 3 vấn đề: Lost Updated, Dirty Read và Unrepeatable Read

❖ Nhược điểm:

- Chưa giải quyết được vấn đề Phantom, do vẫn cho phép insert hoặc delete những dòng dữ liệu thỏa điều kiện thiết lập Slock.
- Slock được giữ đến hết giao tác => cản trở việc cập nhật dữ liệu của các giao tác khác.

Serializable



❖ Đặc điểm:

- **Đọc dữ liệu:** SQL server tự động thiết lập SLock trên đơn vị dữ liệu được đọc và giữ Slock này đến hết giao tác.
- Không cho phép thêm những dòng dữ liệu thỏa mãn điều kiện thiết lập Slock.
- **Ghi dữ liệu:** SQL server tự động thiết lập XLock trên đơn vị dữ liệu được ghi, XLock được giữ cho đến hết giao tác.

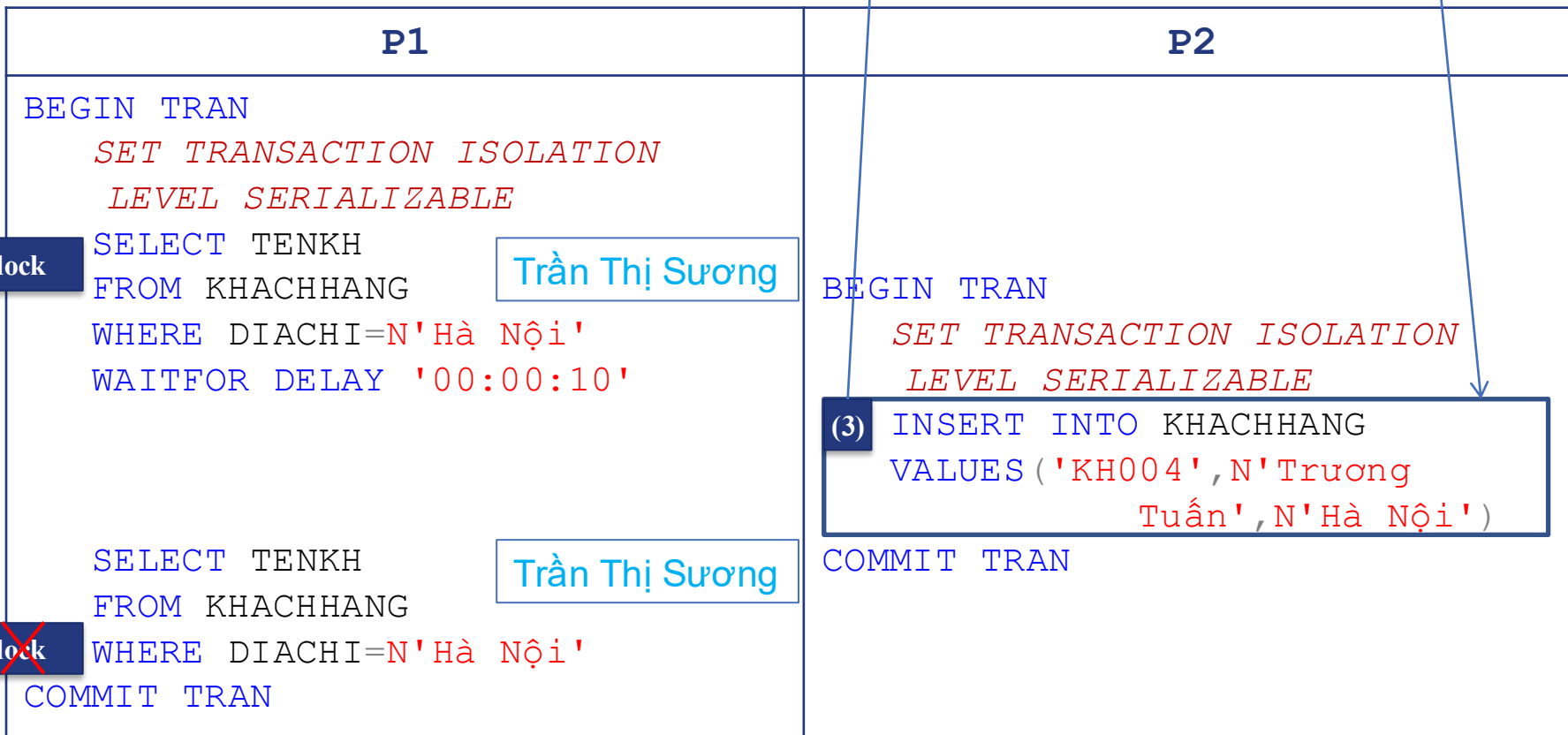
Ví dụ về Serializable

- Repeatabl read => Giải quyết được Phantom

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	HA NOI
KH003	Bùi Văn Toàn	TPHCM

MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM
KH004	Trương Tuấn	Hà Nội

Lệnh INSERT của P2 phải chờ đến khi P1 giải phóng khóa Slock mới được thực hiện



Nhận xét Serializable



❖ Ưu điểm:

- Giải quyết được 4 vấn đề: Lost Updated, Dirty Read, Unrepeatable Read và Phantom

❖ Nhược điểm:

- Slock được giữ đến hết giao tác => cản trở việc cập nhật dữ liệu của các giao tác khác
- Không cho phép Insert/Delete những dòng dữ liệu thỏa mãn điều kiện thiết lập Slock => cản trở việc thêm mới dữ liệu của các giao tác khác

LƯU Ý



- ❖ Mức cô lập mặc định trong SQL Server là Read Committed
- ❖ Mức cô lập không quan tâm khóa Ulock
- ❖ Tầm vực của Isolation level là ở mức connection chứ không phải mức transaction. (khi 1 connection N được đặt mức cô lập X thì X sẽ phát huy hiệu lực trên tất cả các transaction T_i chạy trên N).

KHÓA TRỰC TIẾP TRONG CÂU LỆNH



Cú pháp:

SELECT...

FROM table1 WITH (lock1[,lock2,...])

DELETE FROM table1 WITH (lock1[,lock2,...])

UPDATE table1 WITH (lock1[,lock2,...])

SET...

WHERE...

DEADLOCK

- ❖ Khi xử lý đồng thời, không tránh khỏi việc giao tác này phải chờ giao tác khác.
- ❖ Nếu xảy ra trường hợp hai giao tác chờ lẫn nhau mãi mãi, không giao tác nào có thể hoàn thành được thì ta gọi đó là hiện tượng Dead Lock

P_1	P_2
Lock(A)	
	Lock(B)
Lock(B) <i>Chờ P_2 unlock(B)</i>	
	Lock(A) <i>Chờ P_1 unlock(A)</i>

GIẢI QUYẾT DEADLOCK



- ❖ SQL Server sẽ chọn 1 trong 2 giao tác gây deadlock để hủy bỏ, khi đó giao tác còn lại sẽ được tiếp tục thực hiện cho đến khi hoàn tất.
- ❖ Giao tác bị chọn hủy bỏ là giao tác mà SQL Server ước tính chi phí cho phần việc đã làm được ít hơn giao tác còn lại.

Ví dụ về Deadlock



MAKH	TENKH	DIACHI
KH001	Đỗ Văn Minh	TPHCM
KH002	Trần Thị Sương	Hà Nội
KH003	Bùi Văn Toàn	TPHCM

P1

```
BEGIN TRAN
  UPDATE KHACHHANG WITH (XLOCK)
  SET DIACHI='VUNG TAU'
  WHERE MAKH='KH001'
  WAITFOR DELAY '00:00:10'
```

```
  UPDATE KHACHHANG WITH (XLOCK)
  SET DIACHI='LONG AN'
  WHERE MAKH='KH002'
COMMIT TRAN
```

P2

```
BEGIN TRAN
  UPDATE KHACHHANG WITH (XLOCK)
  SET DIACHI='VUNG TAU'
  WHERE MAKH='KH002'
```

```
  UPDATE KHACHHANG WITH (XLOCK)
  SET DIACHI='DA NANG'
  WHERE MAKH='KH001'
COMMIT TRAN
```



Hết chương 5